

326 一个容器就是一些特定类型对象的集合。顺序容器（sequential container）为程序员提供了控制元素存储和访问顺序的能力。这种顺序不依赖于元素的值，而是与元素加入容器时的位置相对应。与之相对的，我们将在第 11 章介绍的有序和无序关联容器，则根据关键字的值来存储元素。

标准库还提供了三种容器适配器，分别为容器操作定义了不同的接口，来与容器类型适配。我们将在本章末尾介绍适配器。



本章的内容基于 3.2 节、3.3 节和 3.4 节中已经介绍的有关容器的知识，我们假定读者已经熟悉了这几节的内容。

9.1 顺序容器概述

表 9.1 列出了标准库中的顺序容器，所有顺序容器都提供了快速顺序访问元素的能力。但是，这些容器在以下方面都有不同的性能折中：

- 向容器添加或从容器中删除元素的代价
- 非顺序访问容器中元素的代价

表 9.1：顺序容器类型	
vector	可变大小数组。支持快速随机访问。在尾部之外的位置插入或删除元素可能很慢
deque	双端队列。支持快速随机访问。在头尾位置插入/删除速度很快
list	双向链表。只支持双向顺序访问。在 list 中任何位置进行插入/删除操作速度都很快
forward_list	单向链表。只支持单向顺序访问。在链表任何位置进行插入/删除操作速度都很快
array	固定大小数组。支持快速随机访问。不能添加或删除元素
string	与 vector 相似的容器，但专门用于保存字符。随机访问快。在尾部插入/删除速度快

除了固定大小的 array 外，其他容器都提供高效、灵活的内存管理。我们可以添加和删除元素，扩张和收缩容器的大小。容器保存元素的策略对容器操作的效率有着固有的，有时是重大的影响。在某些情况下，存储策略还会影响特定容器是否支持特定操作。

327 例如，string 和 vector 将元素保存在连续的内存空间中。由于元素是连续存储的，由元素的下标来计算其地址是非常快速的。但是，在这两种容器的中间位置添加或删除元素就会非常耗时：在一次插入或删除操作后，需要移动插入/删除位置之后的所有元素，来保持连续存储。而且，添加一个元素有时可能还需要分配额外的存储空间。在这种情况下，每个元素都必须移动到新的存储空间中。

list 和 forward_list 两个容器的设计目的是令容器任何位置的添加和删除操作都很快速。作为代价，这两个容器不支持元素的随机访问：为了访问一个元素，我们只能遍历整个容器。而且，与 vector、deque 和 array 相比，这两个容器的额外内存开销也很大。

deque 是一个更为复杂的数据结构。与 string 和 vector 类似，deque 支持快速

的随机访问。与 `string` 和 `vector` 一样，在 `deque` 的中间位置添加或删除元素的代价（可能）很高。但是，在 `deque` 的两端添加或删除元素都是很快的，与 `list` 或 `forward_list` 添加删除元素的速度相当。

`forward_list` 和 `array` 是新 C++ 标准增加的类型。与内置数组相比，`array` 是一种更安全、更容易使用的数组类型。与内置数组类似，`array` 对象的大小是固定的。因此，`array` 不支持添加和删除元素以及改变容器大小的操作。`forward_list` 的设计目标是达到与最好的手写的单向链表数据结构相当的性能。因此，`forward_list` 没有 `size` 操作，因为保存或计算其大小就会比手写链表多出额外的开销。对其他容器而言，`size` 保证是一个快速的常量时间的操作。

C++
11

新标准库的容器比旧版本快得多，原因我们将在 13.6 节（第 470 页）解释。新标准库容器的性能几乎肯定与最精心优化过的同类数据结构一样好（通常会更好）。现代 C++ 程序应该使用标准库容器，而不是更原始的数据结构，如内置数组。

确定使用哪种顺序容器



通常，使用 `vector` 是最好的选择，除非你有很好的理由选择其他容器。

以下是一些选择容器的基本原则：

- 除非你有很好的理由选择其他容器，否则应使用 `vector`。
- 如果你的程序有很多小的元素，且空间的额外开销很重要，则不要使用 `list` 或 `forward_list`。
- 如果程序要求随机访问元素，应使用 `vector` 或 `deque`。
- 如果程序要求在容器的中间插入或删除元素，应使用 `list` 或 `forward_list`。
- 如果程序需要在头尾位置插入或删除元素，但不会在中间位置进行插入或删除操作，则使用 `deque`。
- 如果程序只有在读取输入时才需要在容器中间位置插入元素，随后需要随机访问元素，则
 - 首先，确定是否真的需要在容器中间位置添加元素。当处理输入数据时，通常可以很容易地向 `vector` 追加数据，然后再调用标准库的 `sort` 函数（我们将在 10.2.3 节介绍 `sort`（第 343 页））来重排容器中的元素，从而避免在中间位置添加元素。
 - 如果必须在中间位置插入元素，考虑在输入阶段使用 `list`，一旦输入完成，将 `list` 中的内容拷贝到一个 `vector` 中。

328

如果程序既需要随机访问元素，又需要在容器中间位置插入元素，那该怎么办？答案取决于在 `list` 或 `forward_list` 中访问元素与 `vector` 或 `deque` 中插入/删除元素的相对性能。一般来说，应用中占主导地位的操作（执行的访问操作更多还是插入/删除更多）决定了容器类型的选择。在此情况下，对两种容器分别测试应用的性能可能就是必要的了。

Best
Practices

如果你不确定应该使用哪种容器，那么可以在程序中只使用 `vector` 和 `list` 公共的操作：使用迭代器，不使用下标操作，避免随机访问。这样，在必要时选择使用 `vector` 或 `list` 都很方便。

9.1 节练习

练习 9.1: 对于下面的程序任务, `vector`、`deque` 和 `list` 哪种容器最为适合? 解释你的选择的理由。如果没有哪一种容器优于其他容器, 也请解释理由。

- (a) 读取固定数量的单词, 将它们按字典序插入到容器中。我们将在下一章中看到, 关联容器更适合这个问题。
- (b) 读取未知数量的单词, 总是将新单词插入到末尾。删除操作在头部进行。
- (c) 从一个文件读取未知数量的整数。将这些数排序, 然后将它们打印到标准输出。



9.2 容器库概览

容器类型上的操作形成了一种层次:

- 某些操作是所有容器类型都提供的 (参见表 9.2, 第 295 页)。
- 另外一些操作仅针对顺序容器 (参见表 9.3, 第 299 页)、关联容器 (参见表 11.7, 第 388 页) 或无序容器 (参见表 11.8, 第 395 页)。
- 还有一些操作只适用于一小部分容器。

329

在本节中, 我们将介绍对所有容器都适用的操作。本章剩余部分将聚焦于仅适用于顺序容器的操作。关联容器特有的操作将在第 11 章介绍。

一般来说, 每个容器都定义在一个头文件中, 文件名与类型名相同。即, `deque` 定义在头文件 `deque` 中, `list` 定义在头文件 `list` 中, 以此类推。容器均定义为模板类 (参见 3.3 节, 第 86 页)。例如对 `vector`, 我们必须提供额外信息来生成特定的容器类型。对大多数, 但不是所有容器, 我们还需要额外提供元素类型信息:

```
list<Sales_data>      // 保存 Sales_data 对象的 list
deque<double>         // 保存 double 的 deque
```

对容器可以保存的元素类型的限制

顺序容器几乎可以保存任意类型的元素。特别是, 我们可以定义一个容器, 其元素的类型是另一个容器。这种容器的定义与任何其他容器类型完全一样: 在尖括号中指定元素类型 (此种情况下, 是另一种容器类型):

```
vector<vector<string>> lines; // vector 的 vector
```

C++
11

此处 `lines` 是一个 `vector`, 其元素类型是 `string` 的 `vector`。



较旧的编译器可能需要在两个尖括号之间键入空格, 例如, `vector<vector<string> >`。

虽然我们可以在容器中保存几乎任何类型, 但某些容器操作对元素类型有其自己的特殊要求。我们可以为不支持特定操作需求的类型定义容器, 但这种情况下就只能使用那些没有特殊要求的容器操作了。

例如, 顺序容器构造函数的一个版本接受容器大小参数 (参见 3.3.1 节, 第 88 页), 它使用了元素类型的默认构造函数。但某些类没有默认构造函数。我们可以定义一个保存这种类型对象的容器, 但我们在构造这种容器时不能只传递给它一个元素数目参数: